GPU computing

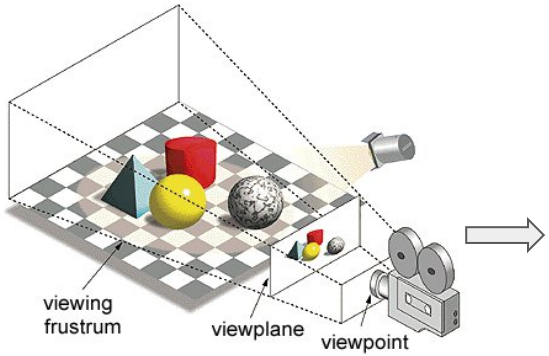
V100, A100, H100 ◂

CUDA ◂

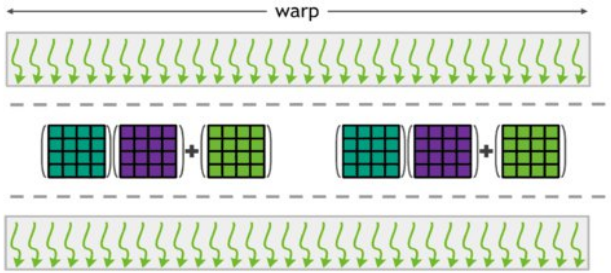
CuDNN ◂

AMP ◂

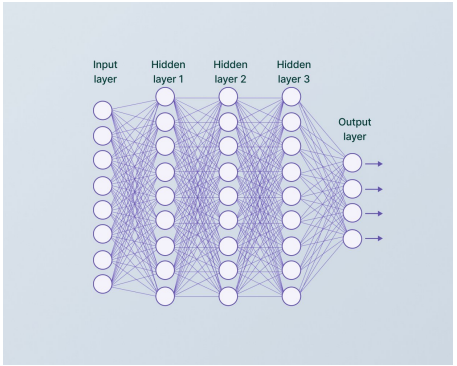
GPU computing



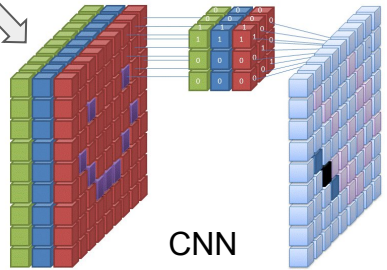
GPU Rendering & Game Graphics Pipeline



Matrix Multiply-accumulate operations

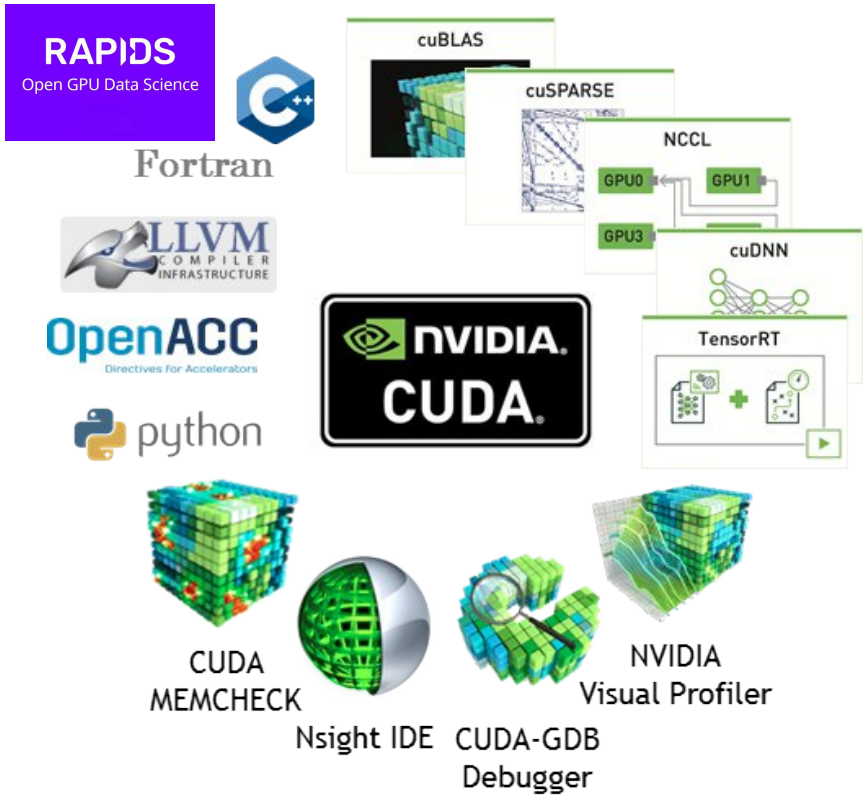


NN



CNN

NVIDIA Galaxy



TRAINING AND INFERENCE

DESKTOP	DATACENTER AND CLOUD
 DGX Spark	 NVL72

INFERENCE AT THE EDGE

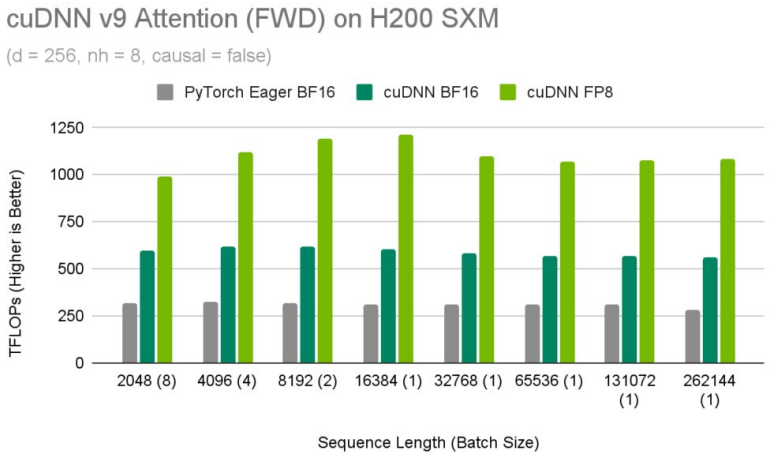
AUTONOMOUS MACHINES	AI SELF-DRIVING PLATFORM
 Jetson Thor	 DRIVE Pegasus

NVIDIA DEEP LEARNING SDK and CUDA

Source : NVidia

CuDNN



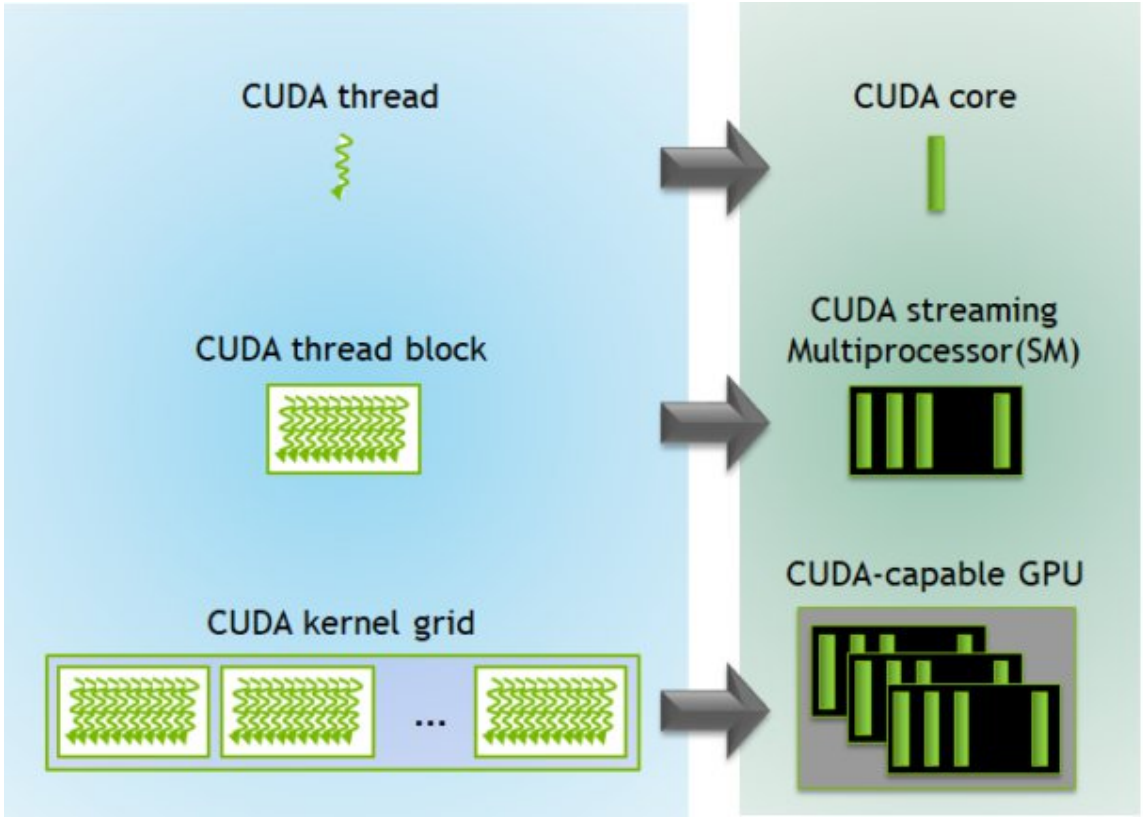


https://developer.nvidia.com/blog/accelerating-transformers-with-nvidia-cudnn-9/

CUDA engineering for deep learning on GPU is handled by cuDNN.

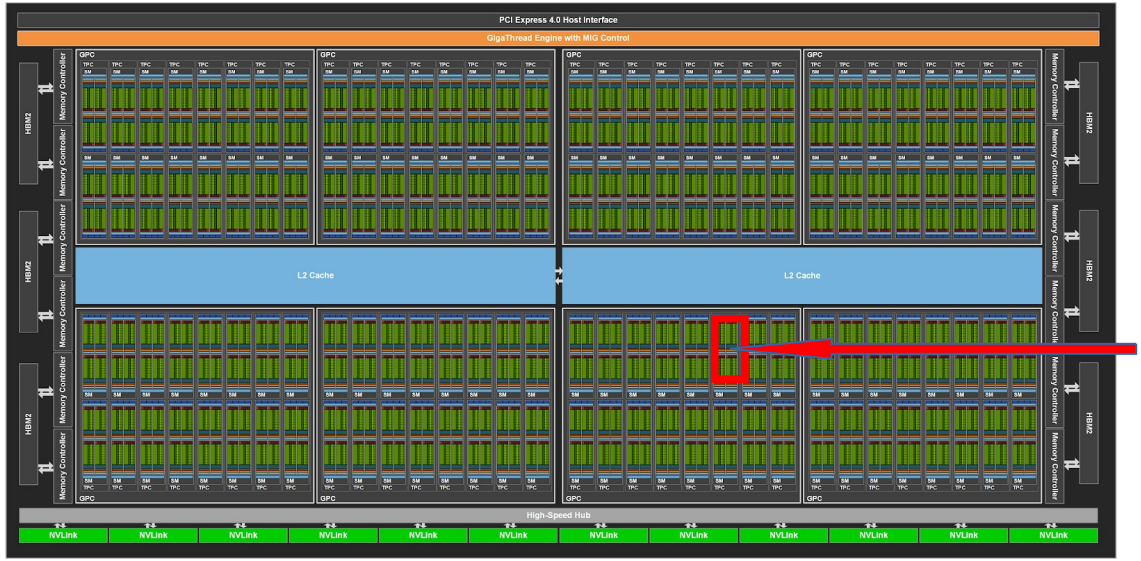
Recommendation: to optimize the use of Tensor Cores and Cuda Cores: Use tensors with dimensions (batch size, sample size, channel, layer dimension, etc.) **multiples of 8**!!

GPU computing : CUDA



Source : NVidia

A100 Architecture

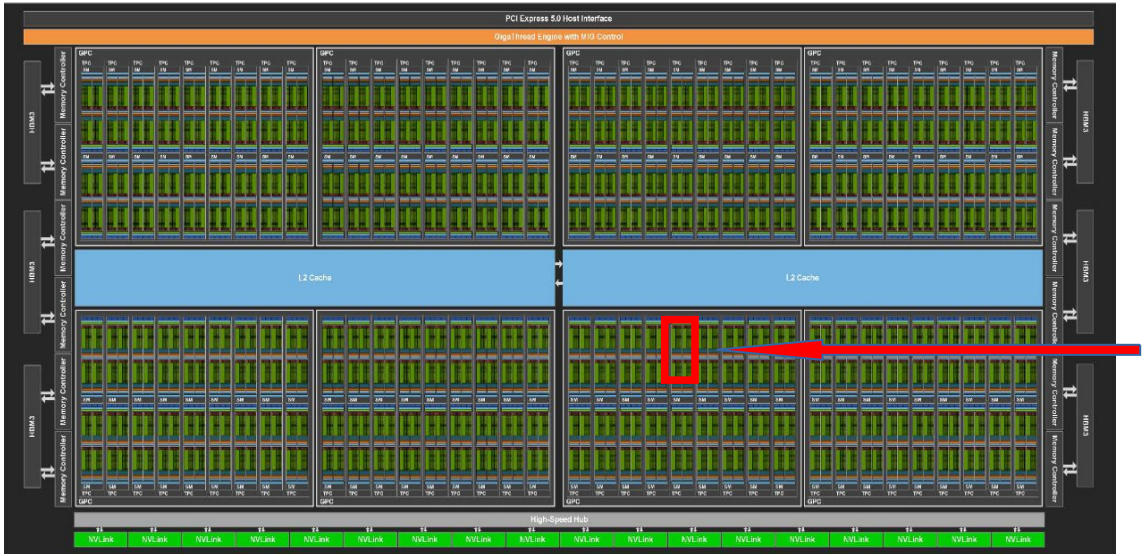




- 8 GPC
- **128** Streaming Multiprocessors (SMs)
- 8192 CUDA Cores
- **512 3eGen** Tensor Cores per full GPU

Source : NVidia

H100 Architecture

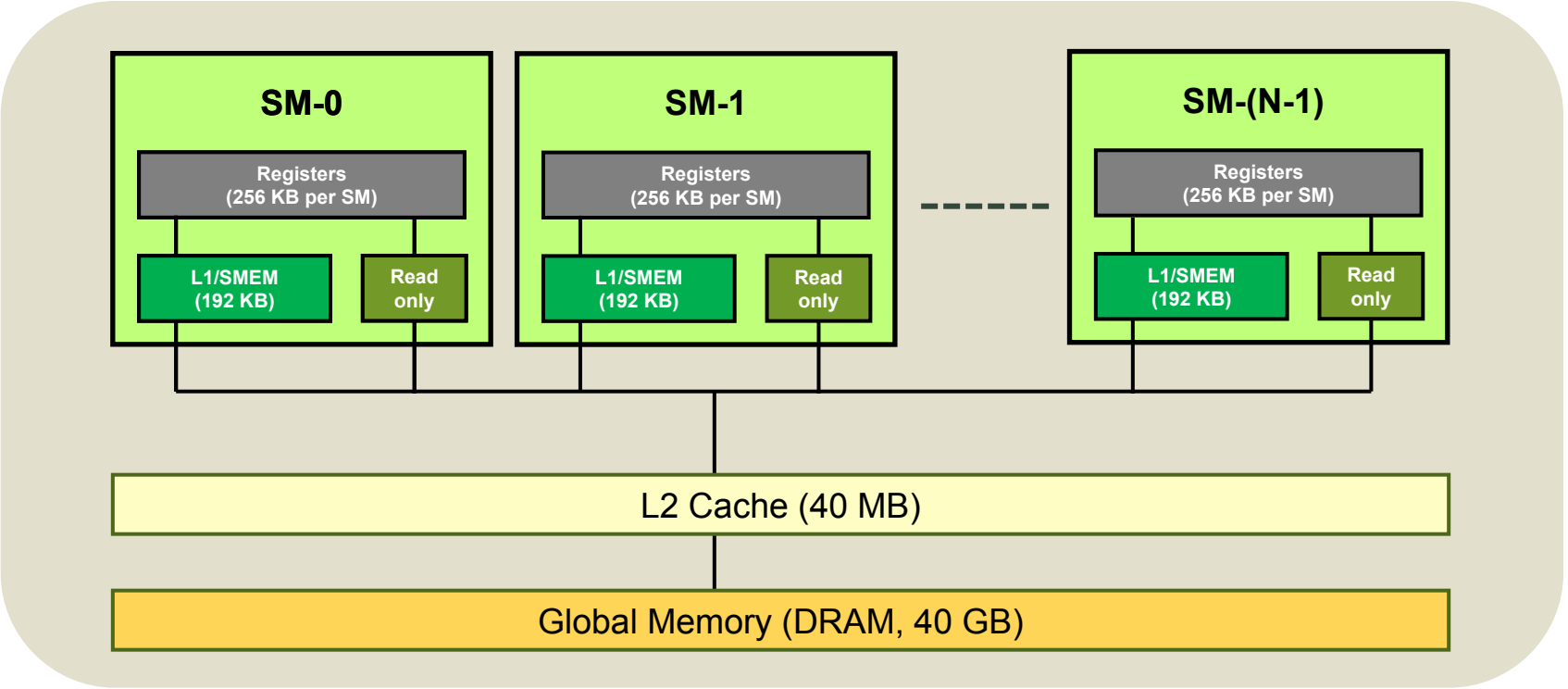




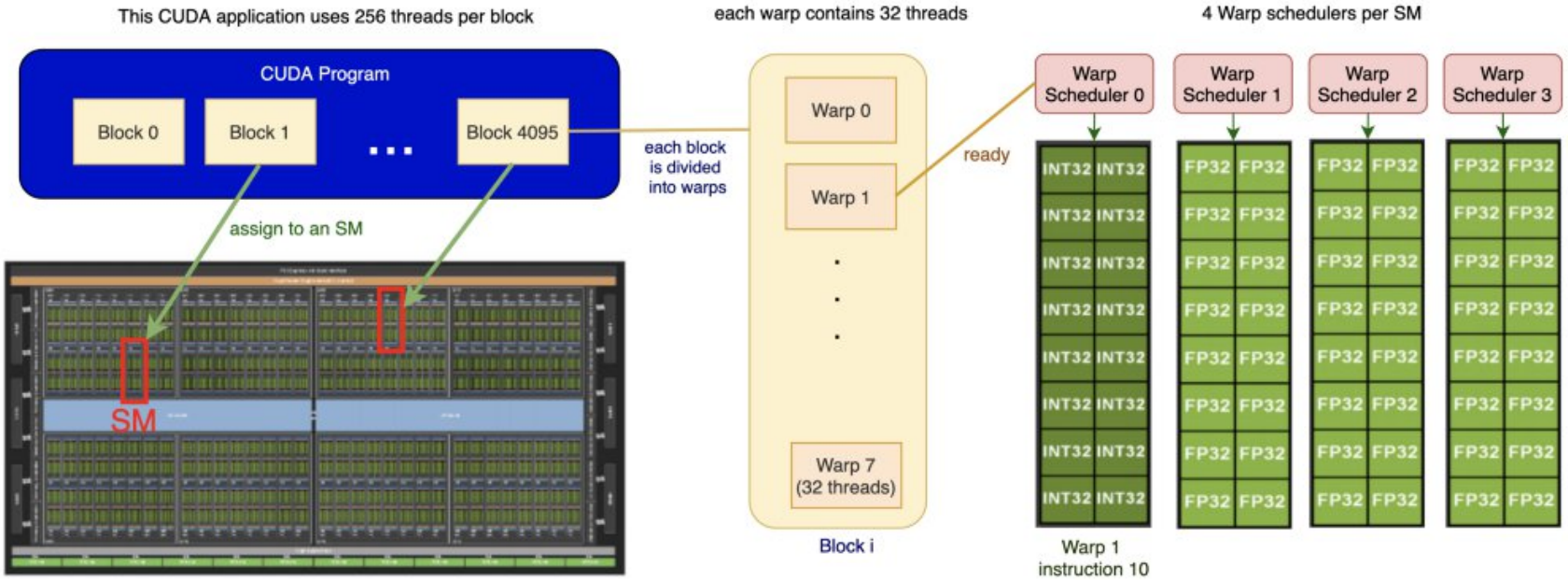
- 8 GPC
- **132** Streaming Multiprocessors (SMs)
- 16896 CUDA Cores
- **528 4eGen** Tensor Cores per H100 SXM5

Source : NVidia

Optimized memory management



CUDA Engineering



Optimization :

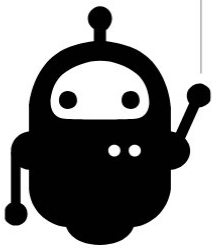
- Block occupancy
- Spread blockss over all SM

Advanced Optimization :

- Kernel Fusion to override initialization times (fewer launches and memory round trips)

TP1 : Accélération GPU





- Envoyer le calcul sur le GPU
- Test Mémoire

Tensor Cores

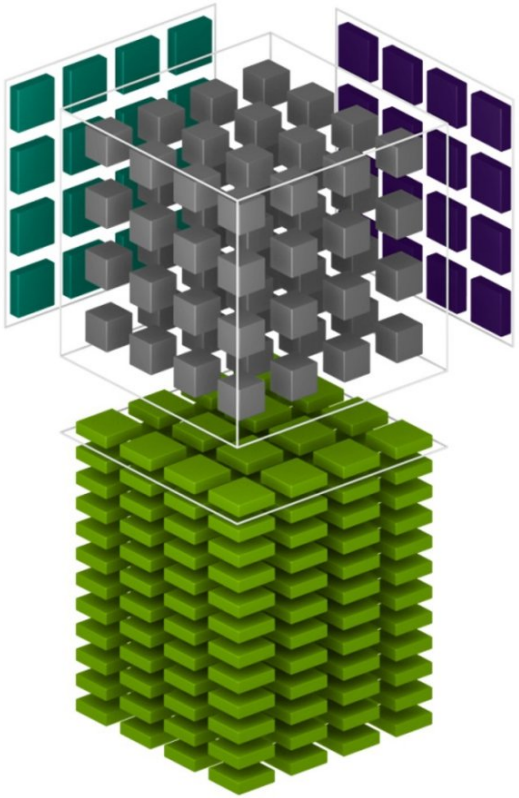
Tensor Cores ◂

Precisions ◂

AMP ◂

Channel last memory format ◂

Tensor Cores



CUDA Core are specialized for **vector computing**.

Tensor Cores are specialized for **matrix calculation**.

$$D = \begin{pmatrix} A_{0,0} & A_{0,1} & A_{0,2} & A_{0,3} \\ A_{1,0} & A_{1,1} & A_{1,2} & A_{1,3} \\ A_{2,0} & A_{2,1} & A_{2,2} & A_{2,3} \\ A_{3,0} & A_{3,1} & A_{3,2} & A_{3,3} \end{pmatrix} \begin{pmatrix} B_{0,0} & B_{0,1} & B_{0,2} & B_{0,3} \\ B_{1,0} & B_{1,1} & B_{1,2} & B_{1,3} \\ B_{2,0} & B_{2,1} & B_{2,2} & B_{2,3} \\ B_{3,0} & B_{3,1} & B_{3,2} & B_{3,3} \end{pmatrix} + \begin{pmatrix} C_{0,0} & C_{0,1} & C_{0,2} & C_{0,3} \\ C_{1,0} & C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,0} & C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,0} & C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

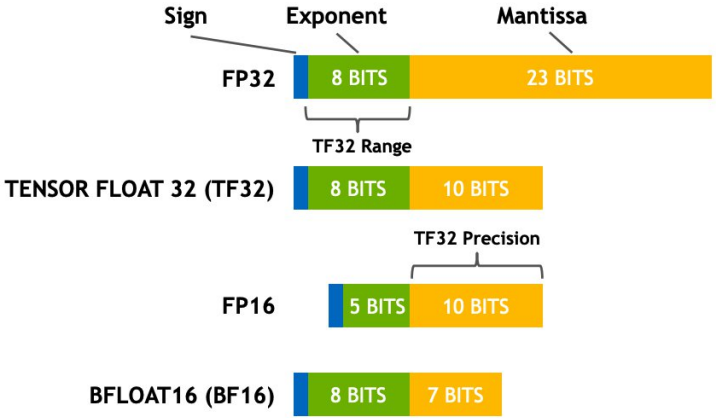
FP16 or FP32 — FP16 — FP16 — FP16 or FP32

Each **V100 Tensor Core** is capable of processing 64 multiply-adds in 1 clock time.

Source : NVidia

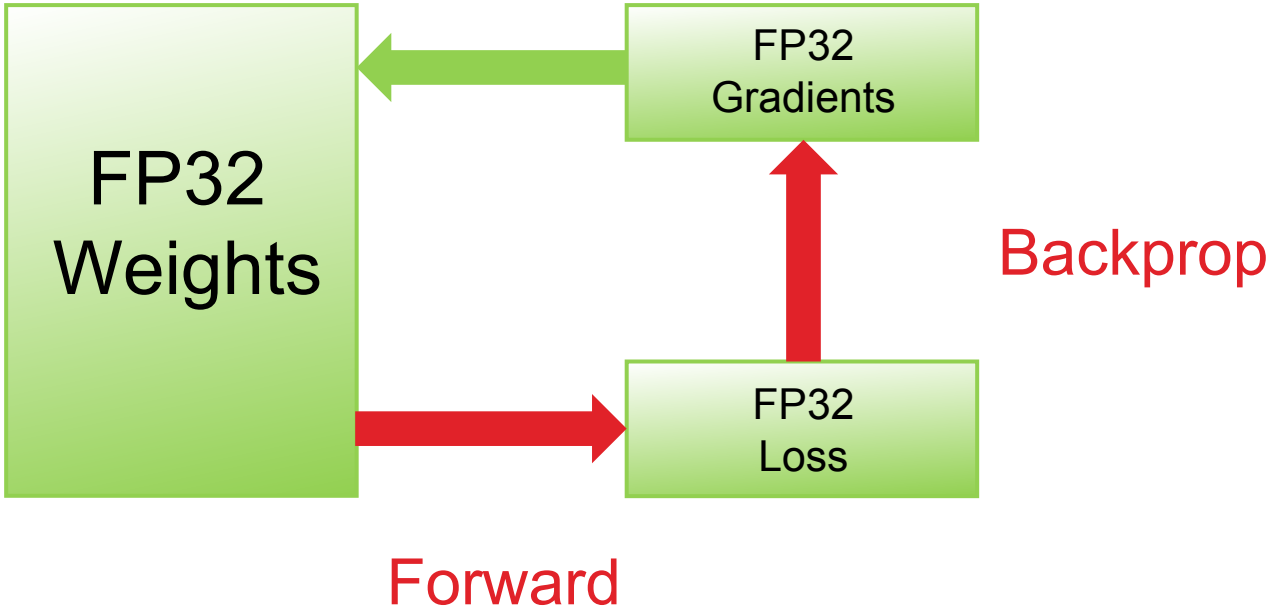
Precisions & Tensor Cores

	NVIDIA H100	NVIDIA A100	NVIDIA Volta
Supported Tensor Core Precisions	**FP8**, FP64, TF32, bfloat16, FP16, ...	FP64, TF32, bfloat16, FP16, INT8, INT4, INT1	FP16
Supported CUDA® Core Precisions	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, bfloat16, INT8	FP64, FP32, FP16, INT8



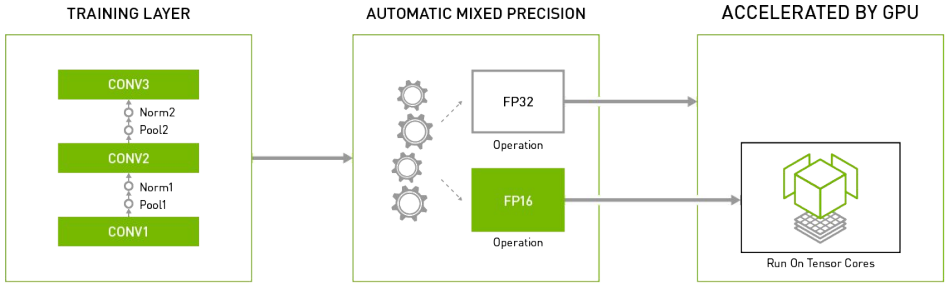
Source : NVidia

Normal Precision Training Loop

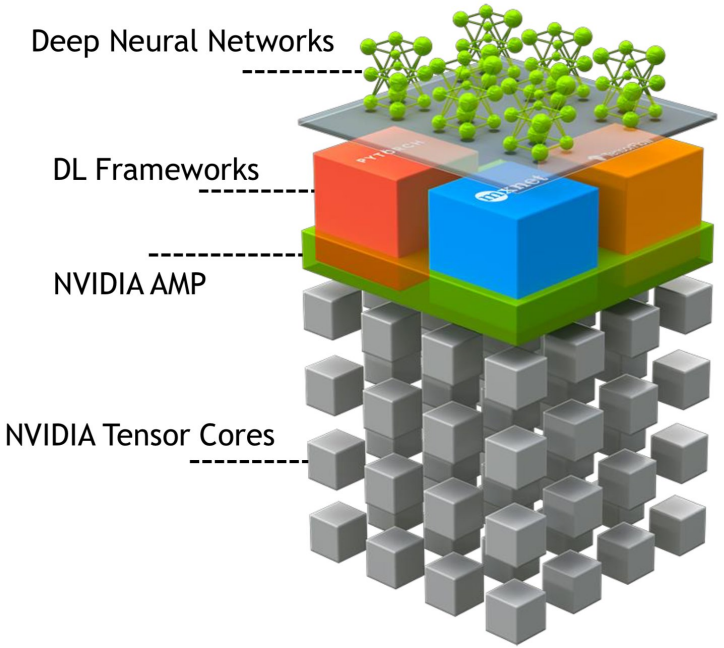


Automatic Mixed Precision

- Automatic Mixed Precision :
  - Necessary with V100 to use Tensor Core
  - The A100s use Tensor Cores with or without MP



- Pros:
  - **Insignificant** loss of precision for model training (gradient, loss, accuracy)
  - **Reduces memory** footprint
  - **Speeds up** calculations
- 2 steps to code :
  - Transforms eligible layers into FP16
  - Uses scaling to calculate gradients

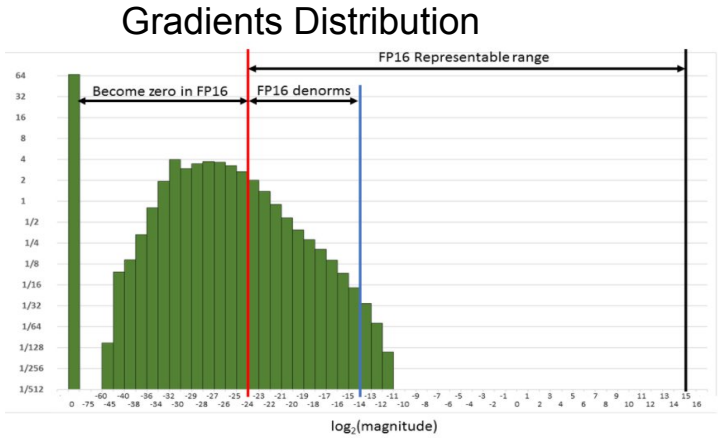


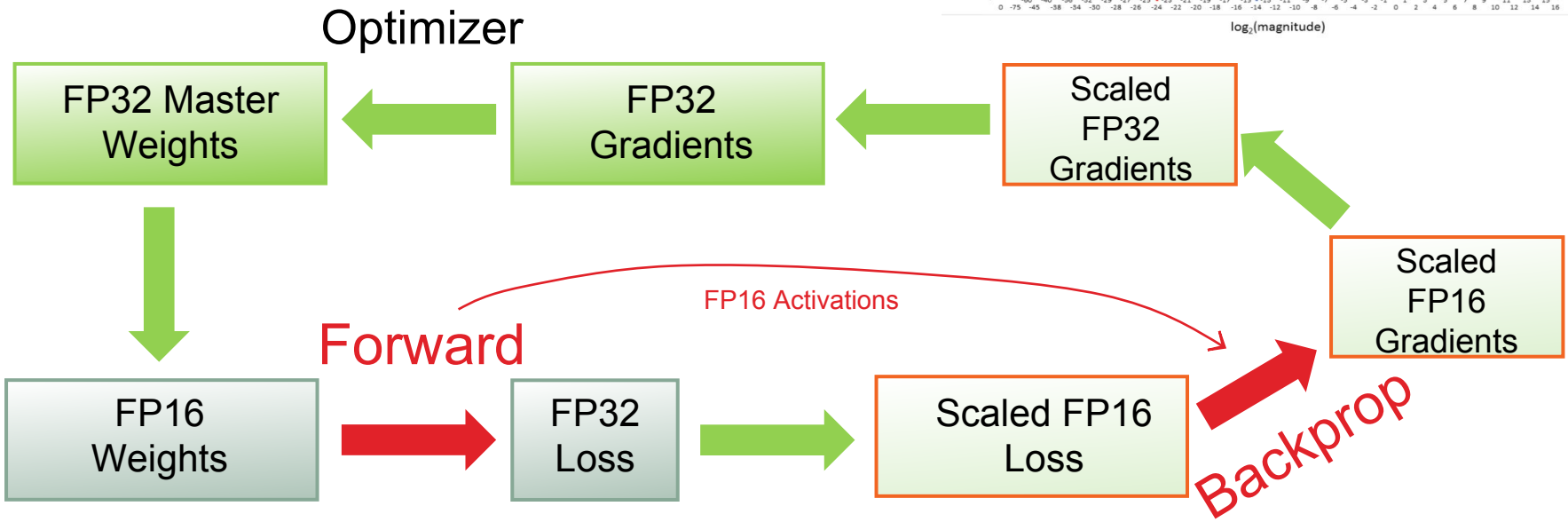
AMP FP16 with Scaler

FP16



In FP16, values lower than 2^{-24} ($5.96e^{-8}$) are considered 0.

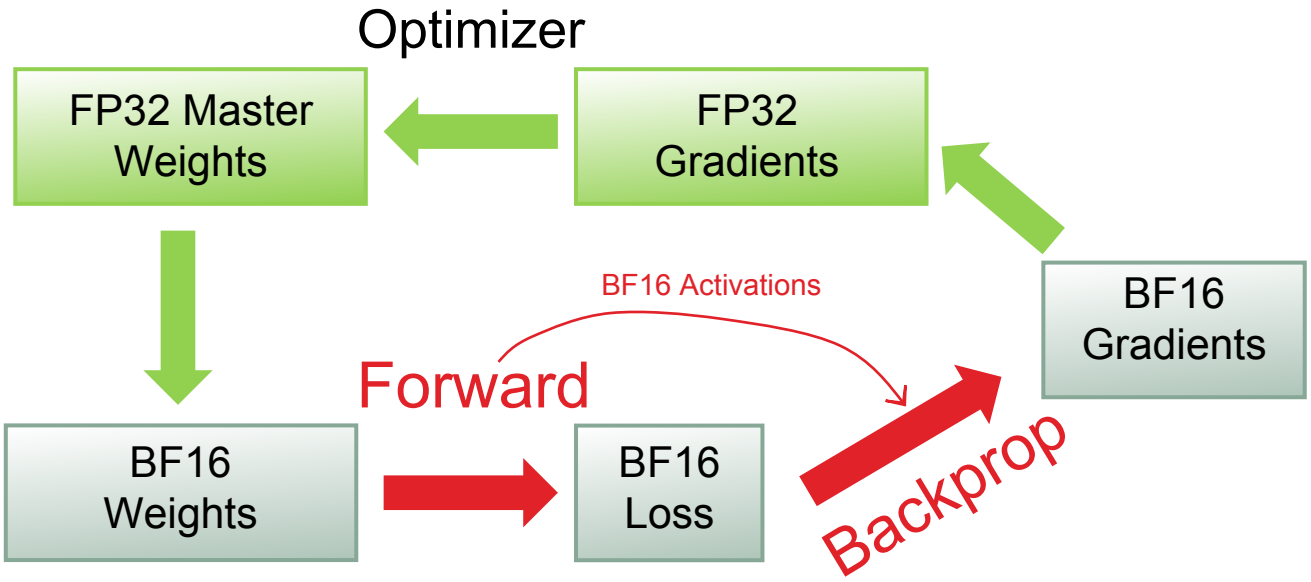




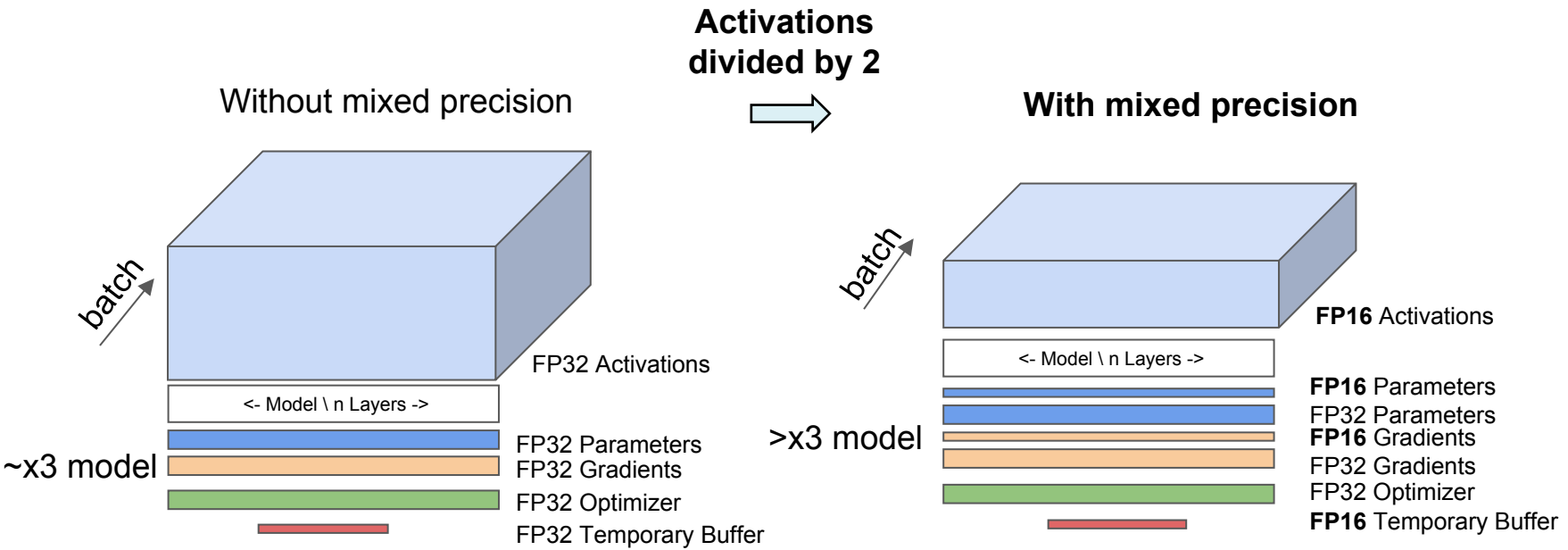
AMP BF16 without Scaler

BF16





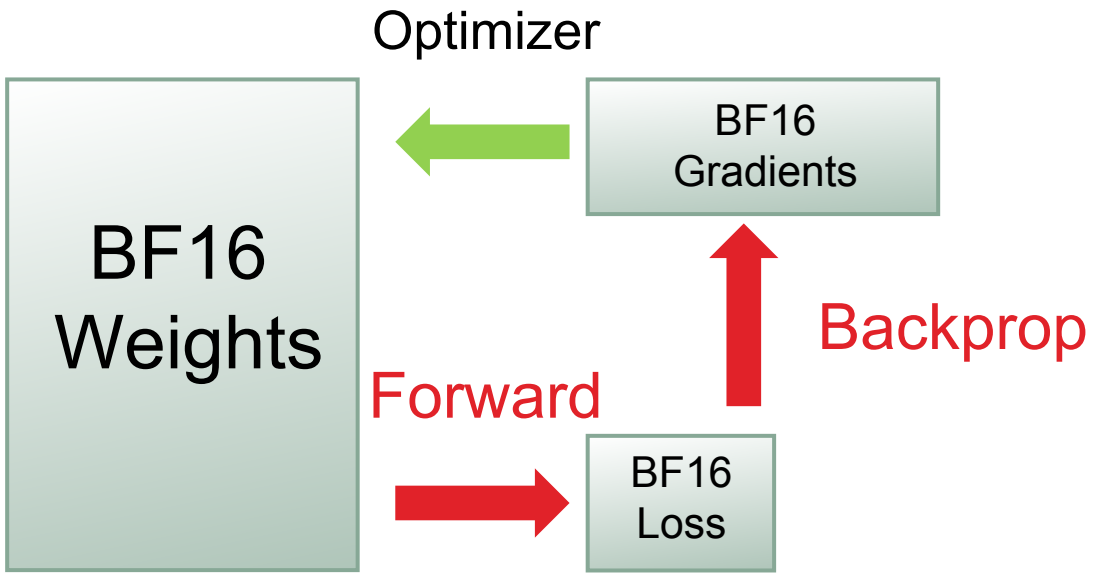
Memory Footprint with Mixed Precision



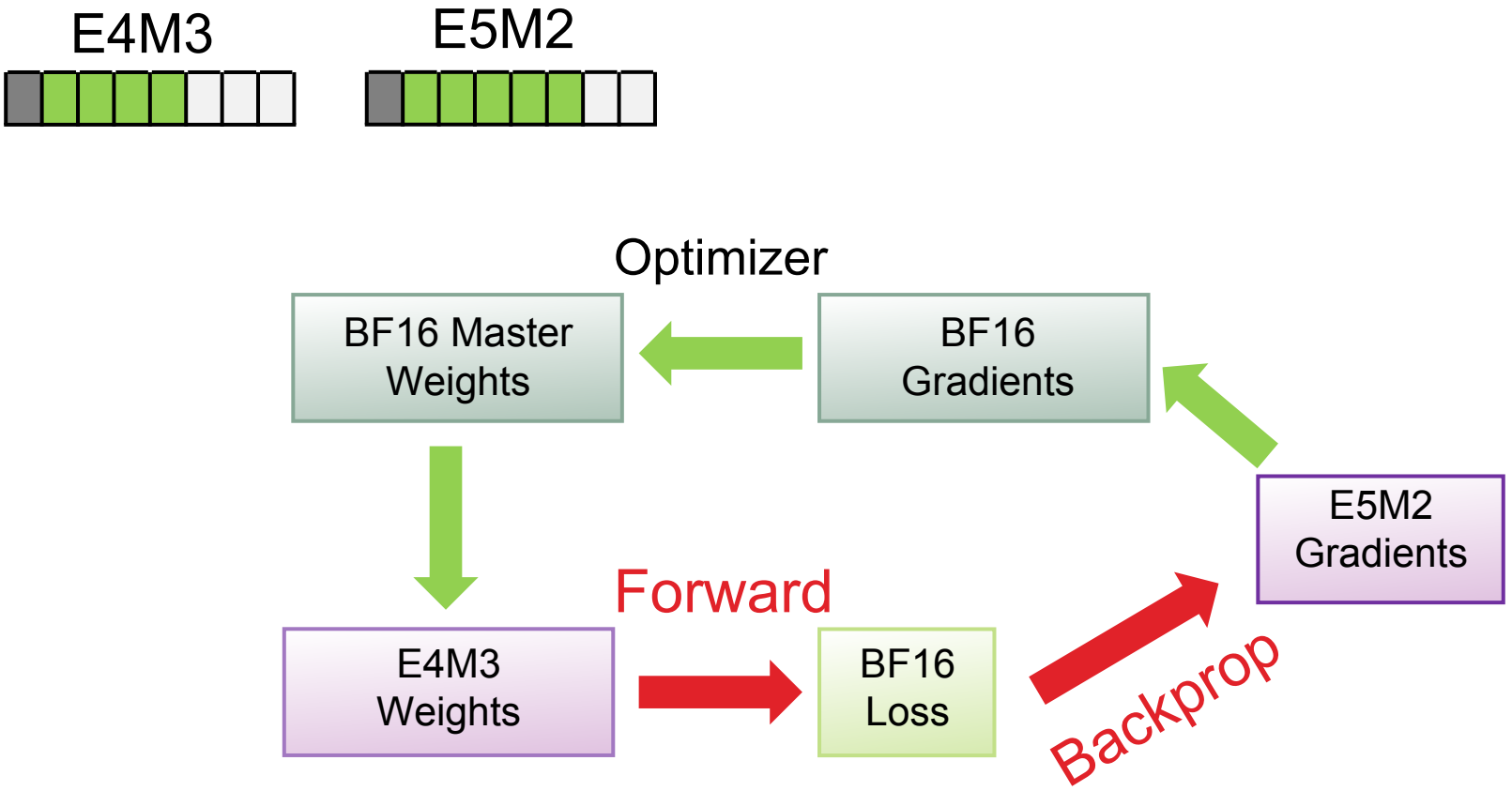
Full Degraded Precision (BF16)

BF16





FP8 Mixed Precision

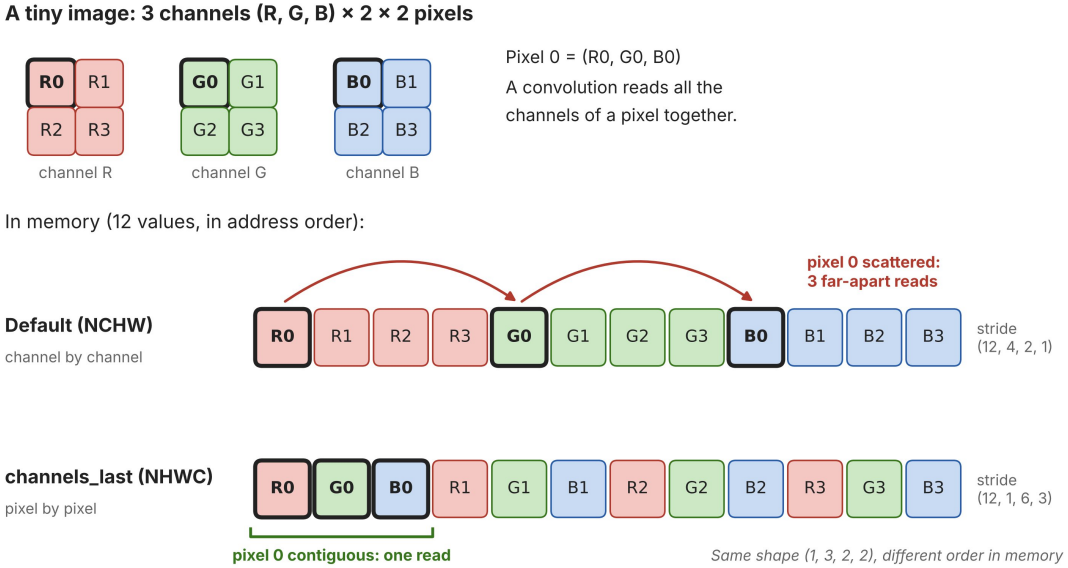


Channel last memory format

batch channel height width

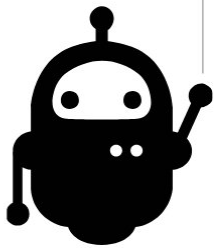
NCHW ← memory contiguity by default

.shape()



TP2&3 : Automatic Mixed Precision





- Activer l’Automatic Mixed Precision
- Test Mémoire
- Activer le channel last memory format